

دانشگاه بوعلی سینا

دانشکده فنی و مهندسی

گروه مهندسی کامپیوتر

تحلیل و طراحی پروژه مازول بیمه تکمیلی

گزارش کار پروژه مهندسی

سامانه تحت وب مدیریت بیمه تکمیلی کارکنان با موتور قانون داده محور، گردش کار
چند مرحله ای و ردیابی کامل

استاد راهنما: جناب آقای دکتر مهدی سخایی نیا

دانشجو: سعید مظاهری

شماره دانشجویی: ۴۰۲۴۲۹۹۰۰۴

رشته و گرایش: مهندسی کامپیوتر – نرم افزار

مرداد ۱۴۰۵

چکیده

با گسترش پوشش بیمه تکمیلی در سازمان‌ها، محاسبه دستی مبلغ قابل پرداخت — با درصد پوشش، سقف هر دفعه، و سقف سالانه — به یکی از منابع اصلی خطا، ابهام، و تأخیر در فرایند رسیدگی به خسارت تبدیل شده است. پروژه بیمه تکمیلی با هدف طراحی و پیاده‌سازی سامانه‌ای هوشمند و تحت وب برای مدیریت درخواست هزینه، محاسبه خودکار مبلغ قابل پرداخت، و ردیابی کامل تصمیم‌ها ارائه می‌دهد. ویژگی متمایزکننده این سامانه آن است که قواعد بیمه‌ای — درصد پوشش، سقف هر دفعه، سقف سالانه، دوره انتظار، و نسبت‌های مجاز خانوار — در کد نوشته نمی‌شوند، بلکه به صورت داده‌ای نسخه‌دار در پایگاه داده نگهداری می‌شوند؛ در نتیجه تغییر سیاست رفاهی سازمان یک عملیات پیکربندی است، نه یک بازانتشار نرم‌افزار.

سامانه با معماری لایه‌ای (دامنه، سرویس، ذخیره‌سازی، انتقال) و بر بستر زبان Go در سمت سرور، پایگاه داده PostgreSQL، و رابط کاربری React با TypeScript به زبان فارسی و راست‌به‌چپ پیاده‌سازی شده است. رابط برنامه‌نویسی سامانه به طور رسمی در قالب یک سند OpenAPI توصیف شده و به عنوان منبع واحد حقیقت میان سرور و کاربر عمل می‌کند. علاوه بر این، موتور گردش کار درخواست، مسیرهای تأیید، رد و بازگرداندن برای تکمیل مدارک را مدیریت می‌کند و هر رویداد مؤثر — به صورت تفکیک‌ناپذیر از خود تغییر داده — در یک لاگ ممیزی عمومی ثبت می‌شود.

درستی سامانه در سه سطح آزمون واحد، یکپارچگی، و سرتاسری روی مرورگر راستی‌آزمایی شده و پنج معیار پذیرش تعریف شده در پیشنهاد پروژه محقق شده‌اند. خروجی نهایی، سامانه‌ای است که ضمن رفع نیاز واقعی یک سازمان، توانمندی پیاده‌سازی استانداردهای مهندسی نرم‌افزار — تفکیک لایه، قرارداد رسمی رابط برنامه‌نویسی، ممیزی اتمیک، و آزمون خودکار در مقیاس‌های کلان — را به اثبات می‌رساند.

واژگان کلیدی

بیمه تکمیلی، موتور قانون داده‌محور، گردش کار چندمرحله‌ای، لاگ ممیزی، کنترل دسترسی مبتنی بر نقش، معماری لایه‌ای، قرارداد رابط برنامه‌نویسی

فهرست مطالب

برای نمایش فهرست، روی آن کلیک کنید و کلید F9 را بزنید.

فصل اول:

کلیات و مقدمه

۱-۱ مقدمه

در سازمان‌هایی که به کارکنان خود بیمه تکمیلی گروهی ارائه می‌دهند، فرایند معمول رسیدگی به خسارت هم‌چنان به صورت دستی و مبتنی بر صفحه‌گسترده انجام می‌شود: کارمند فاکتور را تحویل می‌دهد، کارشناس با ماشین حساب درصد پوشش و دو نوع سقف را با هم ترکیب می‌کند، و نتیجه را جایی یادداشت می‌کند تا به واحد مالی اطلاع دهد. این فرایند نه تنها زمان‌بر است، بلکه هیچ ضمانتی برای صحت محاسبه و امکان بازسازی تصمیم در آینده فراهم نمی‌کند.

پروژه بیمه تکمیلی با هدف رفع این خلأ، به عنوان یک راهکار یکپارچه تحت وب طراحی شده است. این سامانه نه تنها بستری برای دادوستد میان کارمند و کارشناس بررسی است، بلکه به عنوان یک منبع داده معتبر برای مدیر سامانه، ممیز/حسابرس، و در آینده سامانه مادر منابع انسانی نیز عمل می‌کند.

۱-۲ بیان مسئله

کارشناسان بررسی خسارت همواره با چالش «محاسبه دستی و مستعد خطا» و «ابهام در ردیابی تصمیم» مواجه هستند. از دیدگاه مهندسی نرم افزار، مسئله اصلی ساختاری در این حوزه، طراحی موتوری است که بتواند قواعد پوشش بیمه‌ای — که به طور طبیعی در طول زمان تغییر می‌کنند — را بدون نیاز به تغییر کد و بازانتشار نرم افزار در یک ساختار داده‌ای منسجم ذخیره و با سرعت بالا بازخوانی کند، و در عین حال هر تصمیمی که بر اساس آن گرفته می‌شود را به طور غیرقابل جدا شدن از خود تغییر داده، ممیزی کند.

۱-۳ اهمیت و ضرورت پروژه

- کاهش خطای انسانی: حذف فرایندهای سنتی و دستی محاسبه و استعلام قیمت با یک موتور محاسبه یکسان و قابل آزمون.
- شفافیت و ردیابی پذیری: امکان بازسازی این که چه کسی، چه زمانی، و بر اساس کدام نسخه قانون تصمیم گرفته — برای هر رویداد مؤثر در سامانه.
- کاهش هزینه تغییر سیاست: تغییر یک درصد پوشش دیگر به معنای بازآموزی همه و به روزرسانی صفحه‌گسترده‌ها نیست؛ مدیر سامانه خودش این تغییر را اعمال می‌کند.
- مقیاس پذیری: نیاز به زیرساختی که در آینده بتواند صدها یا هزاران کارمند و درخواست هم‌زمان را مدیریت کند، بدون تغییر در معماری اصلی.

۱-۴ اهداف عملیاتی و فنی

۱-۴-۱ اهداف عملیاتی

- ایجاد یک سامانه متمرکز برای ثبت، بررسی، و پرداخت خسارت بیمه تکمیلی کارکنان.

- بهبود تجربه کارمند در ثبت درخواست و پیگیری وضعیت آن و مانده سقف بیمه‌ای.
- تسهیل ارتباط مستقیم میان کارمند، کارشناس بررسی، مدیر سامانه، و ممیز.

۱-۴-۲ اهداف فنی

- پیاده‌سازی معماری استاندارد لایه‌ای (دامنه، سرویس، ذخیره‌سازی، انتقال) با جداسازی کامل قواعد کسب‌وکار از جزئیات وب و پایگاه داده.
- طراحی یک موتور محاسبه سقف بیمه‌ای که هیچ درصد یا سقفی را در کد ثابت نکند.
- پیاده‌سازی امنیت در لایه‌های دسترسی با استفاده از توکن امضاشده و کنترل دسترسی مبتنی بر نقش، در دو سطح (مسیر و مالکیت داده).
- مدیریت هوشمند سطوح دسترسی برای نقش‌های متفاوت (کارمند، کارشناس، مدیر، ممیز).
- انتشار قرارداد رسمی و ماشین‌خوان رابط برنامه‌نویسی (OpenAPI) به‌عنوان منبع واحد حقیقت میان سرور و کاربر.

۱-۵ توصیف سیستم و خروجی پروژه

سامانه بیمه تکمیلی در قالب یک سامانه وب‌محور طراحی شده است که سه سرویس مستقل — پایگاه داده، سرویس برنامه (رابط برنامه‌نویسی)، و رابط کاربری — را با Docker Compose هم‌زمان اجرا می‌کند. از بخش مدیریتی سامانه می‌توان وضعیت کلی، تعداد کارکنان، تعداد درخواست‌ها، قوانین پوشش، و گزارش‌های هزینه را بررسی و نظارت کامل بر روند کار سامانه داشت.

خروجی‌های اصلی سامانه به شرح زیر می‌باشد:

- **سامانه ثبت و رسیدگی به خسارت:** بستری یکپارچه برای ثبت درخواست، بررسی چندمرحله‌ای، محاسبه خودکار مبلغ، و ثبت پرداخت.
- **موتور قانون داده‌محور:** پیاده‌سازی محاسبه پوشش بیمه‌ای مستقل از کد، با نسخه‌بندی زمانی قواعد.
- **لاگ ممیزی جامع:** پیاده‌سازی ثبت اتمیک هر رویداد مؤثر، جهت افزایش کارایی و اعتماد در بازرسی.

۱-۶ محدودیت‌ها و مرزهای پروژه

به‌عنوان یک مهندس نرم‌افزار، مرزهای سیستم را به شکلی تعریف می‌کنیم که از گسترش بی‌رویه جلوگیری شود؛ در ادامه به تشریح این مرزها می‌پردازیم:

- **محدوده:** تمرکز فعلی بر بیمه تکمیلی گروهی سازمانی است، نه بیمه فردی.
- **پرداخت:** درگاه پرداخت واقعی خارج از این محدوده است؛ پرداخت شبیه‌سازی می‌شود و شماره پیگیری تولید می‌کند.
- **یکپارچه‌سازی:** اتصال زنده به سامانه مادر منابع انسانی از طریق یک درگاه با کلید دسترسی فراهم شده، اما اتصال واقعی پیاده نشده است.

- **مقیاس:** هدف ایجاد یک حداقل محصول قابل ارائه با قابلیت توسعه در مقیاس استارت‌آپی است؛ آزمون بار روی حجم بالای داده انجام نشده است.

فصل دوم:

بررسی متون و مطالعات فنی

۲-۱ بررسی مدل‌های کسب‌وکار مشابه

پیش از طراحی، فرایند سنتی رسیدگی به خسارت در سازمان‌های مشابه بررسی شد. روش رایج، مبتنی بر صفحه‌گسترده و بررسی دستی است و مشکلات مشخصی دارد:

مشکل روش دستی	علت
خطای محاسبه	وقتی درصد پوشش و دو نوع سقف با هم درگیر می‌شوند، حساب دستی به راحتی غلط می‌شود.
نامعلوم بودن مانده سقف	کارمند نمی‌داند چقدر از سقف سالانه‌اش باقی مانده، تا وقتی که پیرسد.
غیرقابل ردیابی بودن	پس از مدتی کسی به تصمیمی اعتراض می‌کند؛ هیچ‌کس علت آن تصمیم را به خاطر نمی‌آورد.
کندی فرایند	فاکتور در صف بررسی می‌ماند و کارمند از وضعیت آن بی‌خبر است.
سختی تغییر سیاست	تغییر یک درصد پوشش یعنی بازآموزی همه و به‌روزرسانی همه فایل‌های صفحه‌گسترده.

سامانه بیمه تکمیلی این مدل را با یک گردش کار دیجیتال جایگزین می‌کند: کارمند در سامانه ثبت می‌کند، کارشناس با یک تصمیم آن را پیش می‌برد، سامانه خودش محاسبه و ثبت می‌کند، و همه چیز در لاگ ممیزی قابل بازبینی است.

۲-۲ معماری نرم‌افزار

معماری لایه‌ای (Layered Architecture) به عنوان الگوی معماری اصلی این پروژه انتخاب شده است. این تصمیم به‌طور رسمی در سند تصمیم معماری ADR-۰۰۰۱ ثبت شده و علت آن، رشد ویژگی‌محور نسخه اولیه سامانه بود که در آن، مسیرهای وب مستقیماً پرس‌وجوی پایگاه داده صادر می‌کردند. معماری لایه‌ای در ادامه به‌طور کامل در فصل سوم تشریح می‌شود.

۲-۳ تحلیل فنی تکنولوژی‌های منتخب

بخش	فناوری	دلیل انتخاب
سمت سرور	زبان Go با روتر chi	کارایی بالا، تایپ ایستا، استقرار به‌صورت یک فایل اجرایی واحد
پایگاه داده	PostgreSQL ۱۶	پشتیبانی قوی از تراکنش و قیدهای یکپارچگی داده
دسترسی به داده	GORM برای پرس‌وجو، golang-migrate برای مهاجرت	تفکیک تکامل بایما از منطق خواندن/نوشتن سطرها
احراز هویت	JWT (golang-jwt/v5) و bcrypt	هش امن گذرواژه و توکن بدون حالت با اعتبار محدود
قرارداد رابط برنامه‌نویسی	OpenAPI ۳ (getkin/kin-openapi)	منبع واحد حقیقت و آزمون تطبیق خودکار

بخش	فناوری	دلیل انتخاب
سمت کاربر	۱۹ React با TypeScript و Vite	ساخت رابط تعاملی با ایمنی تایپ و ساخت سریع
مدیریت وضعیت داده	TanStack React Query	کش، واکشی مجدد، و همگام سازی خودکار با سرور
ظاهر رابط	Tailwind CSS و قلم وزیرمتن	پشتیبانی مناسب از چیدمان راست به چپ فارسی
آزمون سرتاسری	Puppeteer (کروم بدون واسط گرافیکی)	شبیه سازی رفتار واقعی کاربر در مرورگر
استقرار	Docker Compose	اجرای یکسان سه سرویس در محیط های مختلف

۲-۴ استانداردهای طراحی

- استفاده از کدهای وضعیت HTTP استاندارد (مانند ۲۰۰، ۲۰۱، ۴۰۰، ۴۰۱، ۴۰۳، ۴۰۹، ۴۲۲) به جای کدهای اختصاصی، تا معنای هر خطا برای هر کلاینت روشن باشد.
- استاندارد سازی قالب تبادل داده: تمام بدنه پیام ها JSON، تمام زمان ها RFC3339، و تمام خطاها به شکل یکسان {"error": "..."} برمی گردند.
- امنیت در تبادل اطلاعات: توکن حامل (Bearer) برای کاربران، و کلید دسترسی جدا (X-API-Key) برای فراخوانی های سامانه به سامانه.

۲-۵ مستند سازی کد و قرارداد رابط برنامه نویسی

بر اساس ADR-۰۰۰۲، رابط برنامه نویسی سامانه به صورت قرارداد-محور طراحی شده است: سند OpenAPI تنها منبع حقیقت است، تایپ های TypeScript سمت کاربر از همان سند تولید می شوند (npm run gen:api)، و دو آزمون خودکار سمت سرور (TestOpenAPIConformance و TestOpenAPISpecCoversEveryRoute) پاسخ های واقعی را با این سند تطبیق می دهند و در صورت وجود مسیری بدون مستند، آزمون را شکست می دهند. تصمیم های معماری مهم دیگر — پول به صورت عدد صحیح ریال (ADR-۰۰۰۳) و زمان کسب و کار پایه تهران با ساعت تزریق شده (ADR-۰۰۰۴) — نیز در قالب همین سبک سند تصمیم معماری ثبت شده اند تا دلیل هر تصمیم فنی برای توسعه دهندگان آینده در دسترس بماند.

۲-۶ معرفی ساختار کلی پروژه

ساختار کلی سامانه در چهار لایه، با وابستگی یک سویه به سمت درون، سازمان یافته است:

```
transport/http  ->  service/*  ->  storage/postgres
                  domain
```


لایه	مسئولیت
دامنه (domain)	موجودیت‌ها و مفاهیم کسب‌وکار به صورت خالص؛ بدون وابستگی به وب یا پایگاه داده
سرویس (service)	منطق کسب‌وکار، تراکنش‌ها و تصمیم‌های دسترسی؛ هفت بسته به تفکیک حوزه
ذخیره‌سازی (storage)	پیاده‌سازی دسترسی به داده؛ تنها لایه‌ای که با کتابخانه نگاشت داده کار می‌کند
انتقال (transport)	مسیرهای وب، اعتبارسنجی ورودی، و نگاشت خطاها به کدهای وضعیت

فصل سوم:

تحلیل و طراحی سیستم

۳-۱ تحلیل مسئله و نیازمندی‌های فنی

در سامانه‌های مدیریت بیمه تکمیلی، یکی از مسائل اصلی، تطبیق صحیح داده‌ها و مقایسه دقیق قواعد پوشش است. سامانه نیاز دارد قواعد ورودی را بر اساس قواعد مشخص پردازش کند، ناهماهنگی‌ها را تشخیص دهد، و نتیجه‌ای دقیق و قابل اعتماد به کاربر ارائه دهد. همچنین از منظر فنی، نیازمندی‌های این لایه عبارت‌اند از:

- پردازش کارآمد درخواست‌های همزمان چند کاربر، بدون تعارض در تراکنش‌های پایگاه داده.
- مدیریت درخواست‌های کاربر در قالب یک ماشین حالت گردش کار صریح و قابل آزمون.
- اطمینان از صحت داده‌ها پیش از ذخیره‌سازی، از طریق قیدهای پایگاه داده (CHECK) و اعتبارسنجی ورودی.
- طراحی منطق قابل توسعه برای ویژگی‌های آینده (مانند بازگذاری مدارک یا اعلان).
- حفظ کارایی در شرایط افزایش حجم داده و درخواست، از طریق نمایه‌گذاری صحیح جداول.

۳-۲ تحلیل نیازمندی‌های عملکردی

نیازمندی‌های عملکردی سامانه بر اساس پنج بازیگر اصلی — کارمند، کارشناس بررسی، مدیر سامانه، ممیز/حسابرس، و سامانه مادر — تحلیل و به شش سناریوی کاربری اصلی (Use Case) دسته‌بندی شده‌اند:

کد	سناریو	بازیگر اصلی
UC-1	ثبت درخواست هزینه توسط کارمند (برای خود یا عضو تحت تکفل)	کارمند
UC-2	بررسی و تصمیم‌گیری کارشناس (تأیید، رد، بازگرداندن برای مدارک)	کارشناس بررسی
UC-3	تغییر قانون پوشش بیمه‌ای — پیکربندی بدون بازانتشار کد	مدیر سامانه
UC-4	بازرسی تاریخچه و جست‌وجوی لاگ ممیزی	ممیز/حسابرس
UC-5	مدیریت کارکنان، اعضای تحت تکفل، و حساب‌های کاربری	مدیر سامانه
UC-6	همگام‌سازی اطلاعات پرسنلی از سامانه مادر	سامانه مادر

سطح مشتری (کارمند) شامل جست‌وجوی وضعیت درخواست‌ها، مشاهده مشخصات فنی پوشش، و ثبت و پیگیری درخواست است؛ سطح کارشناس شامل مدیریت موجودی صف بررسی و قیمت‌گذاری خودکار است؛ و سطح مدیر سامانه شامل مدیریت کاربران و پیکربندی قواعد است.

۳-۳ تحلیل پایگاه داده

همان‌طور که در نمودار موجودیت-رابطه سامانه مشخص است، هر مفهوم اصلی سامانه در قالب یک جدول مستقل ذخیره شده و ارتباط بین جداول از طریق کلیدهای خارجی برقرار می‌شود. این

روش باعث می‌شود داده‌ها به صورت ساخت‌یافته، قابل جست‌وجو و قابل نگهداری باقی بمانند. سامانه در مجموع دوازده جدول اصلی دارد:

نقش در سامانه	جدول
قرارداد سالانه بیمه؛ بالاترین سطح، دارای تاریخ شروع و پایان	insurance_contracts
طرح پوشش زیر یک قرارداد (مثلاً «استاندارد» و «ویژه»)	coverage_plans
فهرست ثابت انواع خدمت قابل ادعا (ویزیت، دارو، دندان‌پزشکی، بستری، عینک)	service_types
قواعد پوشش نسخه‌دار؛ منبع داده‌ای کامل موتور محاسبه	coverage_rules
کارکنان بیمه‌شده؛ وضعیت اشتغال، تاریخ استخدام، طرح تخصیص‌یافته	employees
اعضای تحت تکفل هر کارمند (همسر، فرزند، والد)	dependents
حساب‌های ورود، هر کدام با یک نقش	users
موجودیت مرکزی تراکنشی؛ درخواست هزینه و چرخه بررسی آن	claims
مدارک پیوست درخواست	claim_attachments
یک پرداخت شبیه‌سازی‌شده به‌ازای هر درخواست تأییدشده	payments
لاگ ممیزی عمومی و چندریخت برای همه رویدادهای مؤثر	audit_logs
کلیدهای دسترسی درگاه یکپارچه‌سازی سامانه مادر	integration_api_keys

۳-۴ روابط بین جداول

۳-۴-۱ روابط شاخص موجود در ساختار

- **قرارداد و طرح:** هر insurance_contracts می‌تواند چند coverage_plans داشته باشد (یک‌به‌چند)؛ حذف قرارداد، طرح‌های آن را نیز حذف می‌کند (ON DELETE CASCADE).
- **طرح و قانون پوشش:** coverage_rules به‌ازای هر ترکیب (plan_id, service_type_id) نسخه‌بندی می‌شود؛ یعنی یک طرح می‌تواند چند نسخه قانون در طول زمان داشته باشد.
- **کارمند و عضو تحت تکفل:** هر employees می‌تواند چند dependents داشته باشد؛ relation آن‌ها (spouse, child, parent) با eligible_relations در قانون پوشش تطبیق می‌یابد.
- **درخواست به‌عنوان گره مرکزی:** claims به هر یک از employees, service_types, coverage_plans، و به‌صورت اختیاری dependents متصل است؛ و از سوی دیگر claim_attachments و payments به آن متصل می‌شوند (یک‌به‌چند و یک‌به‌یک به‌ترتیب).
- **لاگ ممیزی چندریخت:** audit_logs.entity_type/entity_id به‌عمد یک کلید خارجی پایگاه‌داده‌ای نیست، بلکه یک ارجاع چندریخت (رشته‌ای) است — تا یک جدول واحد بتواند رویدادهای coverage_rule، claim، و user را به‌طور یکسان ثبت کند، به بهای از دست دادن قید یکپارچگی مرجعی در سطح پایگاه داده.

۳-۵ ارزیابی کیفیت طراحی

۳-۵-۱ تفکیک مناسب دامنه‌ها

داده‌های مرتبط با قرارداد و پوشش، کارکنان، کاربران، درخواست‌ها، پرداخت‌ها، و لاگ ممیزی از هم جدا شده‌اند و هر بخش در جدول‌های مخصوص به خود قرار گرفته است؛ به همین ترتیب در کد نیز هر دامنه به یک بسته سرویس مستقل نگاشت شده است.

۳-۵-۲ قابلیت توسعه

ساختار فعلی به‌گونه‌ای است که می‌توان در آینده ماژول‌های جدید (مانند وام کارکنان یا هزینه آموزش) را با افزودن `service_types` و `coverage_rules` جدید — بدون بازطراحی کامل پایگاه داده — به آن اضافه کرد؛ زیرا هسته «سه‌میه قابل‌پیکربندی + گردش‌کار چندمرحله‌ای + ممیزی کامل» مستقل از حوزه بیمه است.

۳-۵-۳ انسجام روابط

وجود کلیدهای خارجی صریح، قیدهای CHECK بر روی مقادیر شمارشی (نقش، وضعیت اشتغال، نسبت، وضعیت درخواست)، و شمای پایگاه داده‌ای که تنها از طریق مهاجرت‌های نسخه‌بندی‌شده تغییر می‌کند (نه ساخت خودکار جدول توسط کتابخانه نگاشت داده)، انسجام منطقی بین بخش‌های مختلف سامانه را حفظ می‌کند.

۳-۵-۴ نسخه‌بندی قواعد پوشش

`coverage_rules` یک جدول درج‌محور و نسخه‌دار است؛ انتشار قانون جدید هرگز سطر پیشین را ویرایش نمی‌کند، تنها `effective_to` آن را می‌بندد و سطر جدید را درج می‌کند؛ هر دو نوشته در یک تراکنش با یک رکورد ممیزی همراه می‌شوند. «فعال بودن» یک قانون، یک پرس‌وجوی بازه تاریخ است، نه یک پرچم بولی — بنابراین درخواستی که ماه‌ها بعد تأیید می‌شود همچنان بر اساس نسخه قانونی که در تاریخ فاکتور آن فعال بوده، قیمت‌گذاری می‌شود.

فصل چهارم:

شرح عملکرد سامانه

۴-۱ معرفی کلی عملکرد سامانه

سامانه بیمه تکمیلی در قالب یک اپلیکیشن وبمحور طراحی شده است که رفتار هر کاربر را بر اساس نقش او شکل می‌دهد؛ رابط کاربری شامل شانزده صفحه اصلی است و منوی هر کاربر و دکمه‌های فعال در صفحه جزئیات درخواست، تنها بر اساس نقش و وضعیت جاری آن درخواست ساخته می‌شوند — بنابراین کاربر هرگز با گزینه غیرمجاز مواجه نمی‌شود.

۴-۲ نقش‌ها و سطوح دسترسی در سیستم

نقش	توانایی اصلی
مدیر سامانه (admin)	مدیریت کارکنان، اعضای تحت تکفل، کاربران، قراردادهای طرح‌ها، و قواعد پوشش؛ می‌تواند به‌جای هر کارمند درخواست ثبت کند
کارشناس بررسی (reviewer)	بررسی، تأیید (با محاسبه خودکار)، رد (با ذکر دلیل)، بازگرداندن برای مدارک، ثبت پرداخت، و بستن پرونده
کارمند/بیمه‌شده (employee)	ثبت درخواست برای خود یا عضو تحت تکفل، ارسال/ارسال مجدد، مشاهده فقط درخواست‌های خود و سقف باقی‌مانده
ممیز/حسابرس (auditor)	فقط‌خوان: مشاهده هر درخواست و تاریخچه کامل آن، و گزارش‌های مدیریتی؛ بدون امکان تغییر داده

علاوه بر این چهار نقش کاربری، سامانه مادر (منابع انسانی) از طریق یک درگاه جدا با احراز هویت مبتنی بر کلید (X-API-Key) — بدون نشست کاربری — به سامانه متصل می‌شود.

۴-۳ ثبت درخواست هزینه توسط کارمند

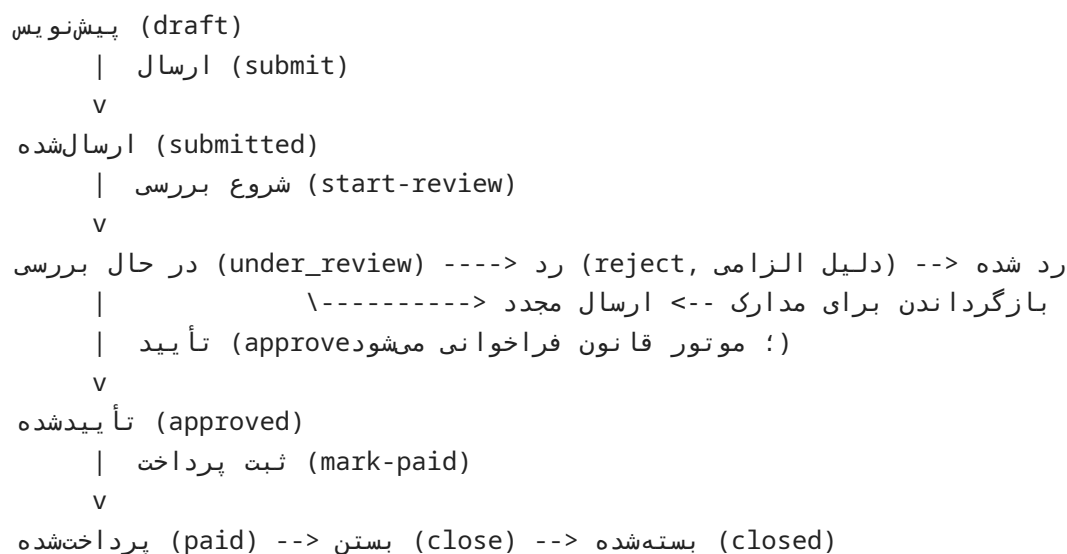
فرآیند ثبت درخواست (UC-1) به شرح زیر است:

- کارمند فرم درخواست جدید را باز می‌کند؛ فهرست انواع خدمت از سامانه بارگذاری می‌شود.
- در صورت ثبت برای یک عضو تحت تکفل، فهرست اعضای تحت تکفل آن کارمند بارگذاری می‌شود.
- کارمند نوع بهره‌بردار، خدمت، مبلغ درخواستی، تاریخ فاکتور، و توضیحات را ارسال می‌کند؛ سرور شناسه کارمند را همیشه به خود کاربر ثابت می‌کند و درخواست با وضعیت «پیش‌نویس» ایجاد می‌شود.
- کارمند پیش‌نویس را بازبینی و برای ارسال نهایی تأیید می‌کند؛ وضعیت به «ارسال‌شده» تغییر می‌کند.

اگر کارمندی طرح پوشش تخصیص‌یافته نداشته باشد، سرور با کد وضعیت ۴۲۲ درخواست ایجاد درخواست را رد می‌کند؛ و اگر کاربری با نقش «کارمند» شناسه کارمند دیگری را در بدنه درخواست وارد کند، سرور آن را نادیده می‌گیرد و همواره شناسه خود کاربر را اعمال می‌کند.

۴-۴ بررسی و تصمیم‌گیری کارشناس

گردش کار کامل یک درخواست (UC-2)، از ثبت تا بستن، در نمودار زیر خلاصه شده است:



هر انتقال وضعیت غیرمجاز (مانند تأیید یک درخواست پیش نویس) با کد وضعیت ۴۰۹ رد می‌شود؛ اقدام یک بازیگر غیرمجاز (مانند تلاش یک کارمند برای شروع بررسی) با کد وضعیت ۴۰۳ رد می‌شود؛ و رد یا بازگرداندن درخواست بدون ذکر دلیل، با کد وضعیت ۴۰۰ رد می‌شود.

۴-۵ موتور محاسبه پوشش و سقف بیمه‌ای

هسته اصلی پروژه، موتور قانونی است که هنگام تأیید یک درخواست، مبلغ قابل پرداخت را در هفت گام محاسبه می‌کند:

- ۱. بارگذاری کارمند و رد درخواست در صورتی که وضعیت اشتغال او «فعال» نباشد.
- ۲. تشخیص نسبت بهره‌بردار: خود کارمند، یا نسبت عضو تحت تکفل پس از تأیید تعلق او به همان کارمند.
- ۳. یافتن قانون پوشش فعال برای ترکیب طرح و نوع خدمت، در تاریخ فاکتور درخواست.
- ۴. بررسی این که نسبت بهره‌بردار در فهرست نسبت‌های مجاز آن قانون قرار دارد.
- ۵. بررسی سپری شدن دوره انتظار از تاریخ استخدام کارمند تا تاریخ فاکتور.
- ۶. جمع مبلغ‌های قابل پرداخت متعهد شده در همان سال قراردادی برای همان کارمند/خدمت/طرح.
- ۷. اعمال درصد پوشش، سپس سقف هر دفعه، سپس مانده سقف سالانه؛ کف‌گیری در صفر و گردکردن نهایی.

سال قراردادی که سقف سالانه بر اساس آن بازنشانی می‌شود، به سالگرد effective_from خود قانون پوشش وابسته است، نه سال میلادی یا شمسی تقویمی؛ و تمام محاسبات، بر پایه منطقه

زمانی تهران و با عدد صحیح ریال انجام می‌شوند تا خطای اعشاری دودویی در جمع‌های مالی انباشته نشود.

۴-۶ مدیریت قوانین پوشش توسط مدیر سامانه

این بخش، مرکز ثقل ادعای اصلی پروژه است (UC-3): مدیر سامانه از صفحه قوانین پوشش، تاریخچه کامل نسخه‌های هر قانون را می‌بیند و می‌تواند نسخه جدیدی با درصد پوشش، سقف هر دفعه، سقف سالانه، دوره انتظار، نسبت‌های مجاز، و تاریخ اعمال جدید ثبت کند. سرور در یک تراکنش، نسخه پیشین فعال همان ترکیب طرح/خدمت را می‌بندد، نسخه جدید را درج می‌کند، و یک رکورد ممیزی با هر دو نسخه (قبل و بعد) ثبت می‌کند — بدون هیچ تغییر کد یا بازانتشار نرم‌افزار.

۴-۷ لاگ ممیزی و بازرسی

ممیز/حسابرس (UC-4) می‌تواند تاریخچه کامل یک درخواست خاص را از صفحه جزئیات آن، یا لاگ ممیزی کل سامانه را با فیلترهای نوع موجودیت، شناسه موجودیت، کاربر عامل، نوع رویداد، و بازه تاریخی جست‌وجو کند. هر رکورد ممیزی، وضعیت قبل و بعد رویداد را در قالب داده ساخت‌یافته (JSON) در خود نگه می‌دارد.

۴-۸ مدیریت کارکنان، اعضای تحت تکفل و کاربران

مدیر سامانه (UC-5) کارکنان را جست‌وجو، ایجاد و ویرایش می‌کند، اعضای تحت تکفل هر کارمند را مدیریت می‌کند، و از صفحه «مدیریت کاربران»، حساب کاربری با نقش مشخص می‌سازد یا نقش/وضعیت فعال‌بودن/گذرواژه یک حساب موجود را تغییر می‌دهد. صفحه «پوشش بیمه‌ای من» برای هر کارمند، سقف هر دفعه، سقف سالانه، مقدار استفاده‌شده، و مانده سالانه را به‌ازای هر نوع خدمت نمایش می‌دهد.

۴-۹ یکپارچه‌سازی با سامانه مادر

درگاه یکپارچه‌سازی (UC-6) دو عملیات دارد: همگام‌سازی گروهی اطلاعات کارکنان بر اساس شماره پرسنلی (ایجاد در صورت جدید بودن، به‌روزرسانی در غیر این صورت، همه در یک تراکنش)، و استعلام وضعیت و مبلغ قابل‌پرداخت یک درخواست مشخص. این درگاه از احراز هویت کاربری کاملاً مستقل است.

۴-۱۰ گزارش‌های مدیریتی

بخش گزارش‌ها (در دسترس مدیر سامانه و ممیز) چهار نمای تجمیعی ارائه می‌دهد:

- خلاصه کلی: تعداد کل درخواست‌ها، مجموع مبلغ پرداختی، تعداد در انتظار بررسی، تأییدشده در انتظار پرداخت، و ردشده.
- هزینه به تفکیک کارمند.
- هزینه به تفکیک نوع خدمت.
- هزینه به تفکیک ماه.

۴-۱۱ توضیح تکیه‌تک صفحات رابط کاربری

صفحه	کارکرد
ورود	احراز هویت با نام کاربری و گذرواژه و دریافت توکن
درخواست‌های من	فهرست درخواست‌های خود کارمند و وضعیت هر یک
کارتابل بررسی	صف درخواست‌های در انتظار بررسی کارشناس
همه درخواست‌ها	فهرست کامل درخواست‌ها، قابل فیلتر بر اساس وضعیت، تاریخ و کارمند (مدیر/ممیز)
ثبت درخواست جدید	فرم ایجاد درخواست برای خود یا عضو تحت تکفل
جزئیات درخواست	اطلاعات کامل، دکمه‌های اقدام مجاز بر اساس وضعیت و نقش، و تاریخچه
پوشش بیمه‌ای من	سقف هر دفعه، سقف سالانه، و مانده هر نوع خدمت برای کارمند
کارکنان	فهرست و جست‌وجوی کارکنان (مدیر/کارشناس)
کارمند جدید / جزئیات کارمند	ایجاد و ویرایش پرونده کارمند، مدیریت اعضای تحت تکفل
قوانین پوشش	تاریخچه نسخه‌های هر قانون و انتشار نسخه جدید (مدیر)
قراردادها / طرح‌های پوشش	مدیریت قراردادهای بیمه و طرح‌های زیرمجموعه
کاربران	مدیریت حساب‌های کاربری و نقش‌ها (مدیر)
گزارش‌ها	داشبورد تجمیعی هزینه و وضعیت (مدیر/ممیز)
تاریخچه اقدامات	لاگ ممیزی قابل جست‌وجو (مدیر/ممیز)
صفحه نیافته	مسیر یافت نشده

فصل پنجم:

تست و ارزیابی

۵-۱ پیاده‌سازی بخش‌های کلیدی

- **ممیزی اتمیک:** نوشتن رکورد لاگ ممیزی و اعمال تغییر بر داده، در یک تراکنش پایگاه داده انجام می‌شوند؛ ساختاراً غیرممکن است که وضعیتی تغییر کند اما ردی از آن ثبت نشود.
- **احراز هویت و کنترل دسترسی:** گذرواژه با bcrypt هش می‌شود، توکن ورود اعتبار هشت‌ساعته دارد، و کنترل دسترسی در دو لایه (نقش در سطح مسیر، مالکیت داده در سطح سرویس) اعمال می‌شود.
- **دقت محاسبات مالی:** مبالغ به صورت عدد صحیح ریال ذخیره و پردازش می‌شوند تا خطای اعشاری دودویی در جمع‌های مالی انباشته نشود؛ تنها یک نقطه گردکردن در کل سامانه وجود دارد.
- **زمان کسب‌وکار:** دوره انتظار و سال قراردادی بر پایه روز تقویمی منطقه زمانی تهران محاسبه می‌شوند، با یک ساعت تزریق‌شده که رفتار آن قابل آزمون است.
- **رابط کاربری فارسی:** شانزده صفحه، کاملاً راست‌به‌چپ، تاریخ‌های هجری شمسی، اعداد فارسی، و حالت‌های نمایش روشن/تیره/خودکار.

۵-۲ تست و ارزیابی سیستم

درستی سامانه در سه سطح راستی‌آزمایی شده است:

سطح آزمون	دامنه	روش
آزمون واحد	ریاضیات موتور قانون، تبدیل‌های مالی، پیکربندی	بدون وابستگی بیرونی؛ اجرای سریع
آزمون یکپارچگی	سرویس‌ها روی پایگاه داده واقعی	هر آزمون در یک تراکنش که در پایان بازگردانده می‌شود
آزمون سرتاسری	کل چرخه از رابط کاربری تا پایگاه داده	هدایت خودکار مرورگر بدون واسط گرافیکی در سیزده گام

علاوه بر این، یک خط لوله یکپارچه‌سازی پیوسته (CI) در هر تغییر، به صورت خودکار بررسی سبک کد (golangci-lint و oxlint)، ساخت پروژه (سمت سرور و سمت کاربر)، مهاجرت و آزمون پایگاه داده، و اجرای کامل آزمون‌ها را انجام می‌دهد؛ آزمون سرتاسری علاوه بر هر push، هر شب نیز به صورت زمان‌بندی‌شده اجرا می‌شود.

۵-۳ سناریوهای آزمون

سناریوهای مختلفی از سامانه که در پوشش تست‌ها و ارزیابی‌ها قرار گرفته است به شرح زیر می‌باشد:

- سناریوی گردش کار کامل: ثبت درخواست توسط کارمند، شروع بررسی، تأیید (با محاسبه خودکار)، ثبت پرداخت، و بستن پرونده.

- سناریوی رد و بازگرداندن: بررسی این که رد یا بازگرداندن بدون دلیل رد می شود، و مسیر بازگرداندن -> ارسال مجدد به درستی کار می کند.
 - سناریوی انتقال نامعتبر: تلاش برای تأیید یک درخواست هنوز پیش نویس، با کد وضعیت ۴۰۹ رد می شود.
 - سناریوی تغییر قانون پوشش: انتشار نسخه جدید یک قانون (مثلاً افزایش پوشش دندان پزشکی از ۵۰ به ۶۵ درصد) و راستی آزمایی این که درخواست بعدی با نرخ جدید محاسبه می شود.
 - سناریوی کنترل دسترسی: تلاش یک بازیگر غیرمجاز (مثلاً کارمند) برای انجام اقدام کارشناس، با کد وضعیت ۴۰۳ رد می شود.
 - سناریوی یکپارچگی داده: بررسی این که داده های ارسالی در بدنه درخواست، دقیقاً مطابق ساختار پایگاه داده ذخیره می شوند.
 - سناریوی تطبیق قرارداد: به صورت عمدی یک اختلاف میان پیاده سازی و سند OpenAPI ایجاد شد تا تأیید شود آزمون تطبیق آن را تشخیص می دهد و شکست می خورد.
- برای صحت تغییر بازنمایی پول از عدد اعشاری به عدد صحیح ریال، نوزده حالت مرزی محاسبه در یک پرونده مرجع ثبت شد، سپس تغییر اعمال و نتیجه مقایسه گردید: سیزده حالت بدون تغییر ماند و در شش حالت باقی مانده، بیشترین اختلاف نیم ریال بود — یعنی تنها همان کسر زیر یک ریال حذف شد.

۵-۴ ارزیابی نتایج در برابر معیارهای پذیرش

معیار پذیرش	وضعیت	شاهد
محاسبه درست مبلغ پرداختی و مانده سقف برای حداقل پنج نوع خدمت با درصد و سقف های متفاوت	محقق شد	آزمون های واحد موتور قانون و پرونده مرجع نوزده حالت مرزی
اعمال تغییر قانون پوشش صرفاً از طریق پیکربندی و بدون تغییر کد	محقق شد	صفحه قوانین پوشش و گام اختصاصی در آزمون سرتاسری
اجرای کامل گردش کار شامل مسیرهای تأیید، رد و بازگشت	محقق شد	آزمون های یکپارچگی گردش کار و آزمون سرتاسری
ثبت کامل رویدادها در لاگ ممیزی و امکان بازسازی تاریخچه	محقق شد	صفحه تاریخچه اقدامات و آزمون ثبت رویدادهای چرخه کامل
اعمال درست محدودیت دسترسی بر اساس نقش	محقق شد	آزمون های دسترسی در سطح سرویس و سرتاسری

۵-۵ اندازه پیاده سازی

شاخص	مقدار	شاخص	مقدار
کد سمت سرور	حدود ۵'۱۰۰ خط	مسیرهای رابط برنامه نویسی	۳۰ مسیر / ۳۸ عملیات
کد آزمون سمت سرور	حدود ۱'۱۰۰ خط	جداول پایگاه داده	۱۲
کد سمت کاربر	حدود ۴'۰۰۰ خط	صفحات رابط کاربری	۱۶
نقش های کاربری	۴	وضعیت های درخواست	۹

مقدار	شاخص	مقدار	شاخص
۱۰	انواع رویداد ممیزی	۵	انواع خدمت پیش فرض

فصل ششم:

مدیریت پروژه و برنامه‌ریزی

۶-۱ متدولوژی توسعه نرم‌افزار

در این پروژه از رویکردی تکاملی و مبتنی بر فازهای مشخص استفاده شده که هر فاز با یک هدف فنی روشن، به صورت مستقل قابل تأیید بوده است:

فاز	هدف	تاریخ
پایه	ساخت نسخه اولیه سامانه: سرور Go/PostgreSQL، رابط کاربری فارسی راست‌به‌چپ، استقرار Docker، مجموعه آزمون سرتاسری	۱۴۰۵/۰۴/۲۸
بازآرایی معماری (فازهای ۰ تا ۲)	استقرار ابزار و یکپارچه‌سازی پیوسته، و بازآرایی معماری به لایه‌های دامنه/سرویس/ذخیره‌سازی/انتقال	۱۴۰۵/۰۴/۲۸
فاز ۳ — قرارداد رابط برنامه‌نویسی	سند OpenAPI به عنوان منبع واحد حقیقت رابط برنامه‌نویسی	۱۴۰۵/۰۵/۰۶
فاز ۴ — پول و زمان	مهاجرت پول به عدد صحیح ریال و پایه‌گیری زمان کسب‌وکار بر منطقه زمانی تهران	۱۴۰۵/۰۵/۰۶
افزودن آزمون سرتاسری و React Query	افزودن آزمون سرتاسری Puppeteer و یکپارچه‌سازی مدیریت وضعیت داده در رابط کاربری	۱۴۰۵/۰۵/۱۵
مستندسازی	افزودن راهنمای ارائه، شرح محصول، و راهنمای کاربری فارسی	۱۴۰۵/۰۵/۱۵
گزارش نهایی	تدوین گزارش پروژه پایانی به صورت سند Word	۱۴۰۵/۰۵/۱۹

این توالی فازبندی، به جای یک چرخه واحد آب‌شاری، مشابه رویکرد چابک (Agile) با افزایش‌های کوچک و قابل‌تأیید عمل کرده است؛ هر فاز پیش از فاز بعدی، با آزمون‌های خودکار موجود راستی‌آزمایی شده است.

۶-۲ ابزارهای مدیریت و کنترل کیفیت

- **Git و کنترل نسخه:** مدیریت کد، کنترل تغییرات، و امکان همکاری تیمی در نسخه‌های آتی.
- **GitHub Actions (یکپارچه‌سازی پیوسته):** اجرای خودکار بررسی سبک کد، ساخت، مهاجرت پایگاه داده آزمون، و اجرای آزمون‌ها در هر push و هر شب برای آزمون سرتاسری.
- **golangci-lint و oxlint:** بررسی سبک و کیفیت کد سمت سرور (Go) و سمت کاربر (TypeScript).
- **openapi-typescript:** تولید خودکار تایپ‌های TypeScript از سند OpenAPI، برای جلوگیری از واگرایی مستندات از پیاده‌سازی.

۶-۳ استانداردهای کدنویسی و کیفیت

- رعایت اصول طراحی لایه‌ای در تمامی لایه‌های مختلف جهت کاهش وابستگی‌ها و افزایش انعطاف‌پذیری.
- استاندارد نام‌گذاری: هر بسته سرویس بر اساس حوزه کسب‌وکار خود نام‌گذاری شده (auth, employee, claim, coverage, rule, audit, report, integration).
- کدنویسی مستند: هر تصمیم معماری مهم در یک سند تصمیم معماری (ADR) با زمینه، گزینه‌های بررسی‌شده، و پیامدها ثبت شده است.
- بازیابی دوره‌ای کدها: جهت شناسایی کدهای تکراری و ارتقای بهینگی الگوریتم‌ها، از جمله ادعای صریح در کد موتور قانون مبنی بر عدم درج هیچ درصد یا سقفی در GO.

۶-۴ مدیریت دانش و مستندسازی فنی

مستندات پروژه در پوشه docs/ نگهداری می‌شوند و هر سند هدف مشخصی دارد:

محتوا	سند
معماری سامانه و تصمیم‌های طراحی لایه‌ای	ARCHITECTURE.md
نمودار موجودیت-رابطه و شرح جدول به جدول	ERD.md
سناریوهای کاربری اصلی و جریان‌های جایگزین/خطا	USE-CASES.md
شرح انسانی رابط برنامه‌نویسی؛ سند مرجع خود OpenAPI است	API-CONTRACT.md
قرارداد رسمی و ماشین‌خوان رابط برنامه‌نویسی	backend/api/openapi.yaml
تصمیم‌های معماری: لایه‌بندی، قرارداد رابط برنامه‌نویسی، پول صحیح، زمان کسب‌وکار	adr/0001 تا 0004
شرح محصول به زبان ساده برای مخاطب غیرفنی	PRODUCT-FA.md
متن ارائه و راهنمای کاربری صفحه به صفحه به فارسی	PRESENTATION-FA.md / USER-JOURNEY-FA.md
برنامه بازآرایی معماری در فازهای ۰ تا ۴	REFACTORING-PLAN.md

۶-۵ شاخص‌های عملکرد کلیدی در پروژه

- **نرخ خطا کشف‌شده:** تعداد باگ‌های کشف‌شده در هر مازول که با انجام تست‌های هر واحد واحد به حداقل رسید.
- **پوشش مسیرهای رابط برنامه‌نویسی:** هر مسیر سرویس‌دهی شده باید در سند OpenAPI مستند باشد؛ در غیر این صورت آزمون خودکار (TestOpenAPISpecCoversEveryRoute) شکست می‌خورد — یعنی پوشش صددرصدی مستندسازی به صورت اجباری تضمین می‌شود.
- **سرعت تحویل:** توانایی تیم در تکمیل فازهای تعریف‌شده (پایه، بازآرایی، قرارداد رابط برنامه‌نویسی، پول و زمان) در بازه‌های زمانی مشخص.

- **پایداری سیستم:** اطمینان از عملکرد صحیح سرویس‌ها در شرایط تست با پایگاه داده واقعی — هر آزمون یکپارچگی در یک تراکنش اجرا و در پایان بازگردانده می‌شود تا داده مرجع دست‌نخورده بماند.

فصل هفتم:

نتیجه‌گیری و پیشنهادات

۷-۱ دستاوردهای پروژه

- **ایجاد یک پایگاه داده متمرکز:** جمع‌آوری اطلاعات قراردادهای طرح‌ها، کارکنان، درخواست‌ها و ممیزی در یک سامانه واحد.
- **موتور قانون داده‌محور:** امکان تغییر سیاست رفاهی سازمان بدون هیچ تغییر کد یا بازانتشار نرم‌افزار.
- **گردش کار چندمرحله‌ای و قابل ردیابی:** پیاده‌سازی کامل مسیرهای تأیید، رد، و بازگرداندن، با ممیزی اتمیک هر رویداد.
- **قرارداد رسمی رابط برنامه‌نویسی:** سند OpenAPI به عنوان منبع واحد حقیقت، با آزمون خودکار تطبیق.
- **دقت مالی و زمانی:** محاسبات مالی با عدد صحیح ریال، و زمان کسب و کار پایه منطقه زمانی تهران.
- **قابلیت مقیاس‌پذیری:** طراحی سامانه به گونه‌ای است که می‌تواند در آینده تعداد کاربران و تراکنش‌ها را بدون تغییر در ساختار اصلی، مدیریت کند.

۷-۲ محدودیت‌های پروژه

- عدم اتصال به درگاه‌های پرداخت آنلاین واقعی (در این نسخه صرفاً فرآیند ثبت درخواست پرداخت انجام می‌شود و شماره پیگیری شبیه‌سازی شده تولید می‌گردد).
- عدم استفاده از سیستم‌های هوشمند هوش مصنوعی برای بارگذاری و تشخیص خودکار مدارک پیوست از روی تصویر.
- تمرکز بر بیمه تکمیلی گروهی سازمانی و عدم پوشش بیمه فردی یا ماشین‌آلات صنعتی سنگین.
- عدم انجام آزمون بار روی حجم بالای کاربران و درخواست‌های همزمان.

۷-۳ پیشنهادات آینده

- پیاده‌سازی سیستم بارگذاری مدارک پیوست: استفاده از رابط بارگذاری فایل برای امکان پیوست تصویر فاکتور توسط کارمند.
- افزودن سامانه اعلان (ایمیل/پیامک): اطلاع‌رسانی خودکار تغییر وضعیت درخواست به کارمند، بدون نیاز به سربرزی دستی.
- اتصال به سیستم‌های انباردار: ایجاد اتصال مستقیم و زنده بین سامانه مادر منابع انسانی و سامانه بیمه تکمیلی برای به‌روزرسانی آنی اطلاعات کارکنان.
- توسعه گردش کار چندسطحی تأیید: افزودن سطح تأیید مدیر در کنار کارشناس، برای سازمان‌هایی که چنین رویه‌ای دارند.

- احراز هویت دوعاملی: با توجه به حساسیت داده‌های درمانی، برای افزایش امنیت ورود کاربران توصیه می‌شود.
- بارگذاری گروهی کارکنان از فایل: برای ورود اولیه داده و انتقال از سامانه پیشین.
- توسعه هسته سامانه به سایر خدمات رفاهی: از آن‌جا که هسته «سه‌میه قابل‌پیکربندی + گردش‌کار چندمرحله‌ای + ممیزی کامل» مستقل از حوزه بیمه است، همان هسته برای وام کارکنان، هزینه آموزش، و هزینه سفر نیز قابل استفاده است.

۷-۴ نتیجه نهایی

در نهایت می‌توان نتیجه گرفت که سامانه بیمه تکمیلی توانسته است به هدف اصلی خود دست یابد، یعنی طراحی یک بستر تخصصی برای مدیریت و تسهیل ارتباط میان کارمند، کارشناس بررسی، و مدیر سامانه، دست یابد. این سامانه با تکیه بر معماری مناسب، ساختار داده‌ای منسجم و نسخه‌دار، و مکانیزم‌های امنیتی استاندارد (احراز هویت با توکن، کنترل دسترسی دوسطحی، و ممیزی اتمیک)، قابلیت تبدیل شدن به یک محصول نرم‌افزاری قابل استفاده در محیط واقعی را داراست. همچنین چارچوب طراحی شده برای این پروژه می‌تواند مبنایی برای توسعه‌های آتی و افزودن امکانات پیشرفته‌تر (مانند وام کارکنان، هزینه آموزش، و هزینه سفر) باشد.